Qualcomm Technologies, Inc.

SW6100 Sensors/Sensing Hub Overview

80-74841-90 AA

June 26, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

Confidential - Qualcomm Technologies, Inc. and/or its affiliated companies - May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Contents

1 Sensors Architecture **3**

2 Sensors features **5**
2.1 LPAI processing engines . . . 5
2.2 Sensing hubs . . . 7
2.3 eNPU . . . 8
2.4 Ambient WM . . . 8
2.5 Software features . . . 8
2.6 Rotary side button . . . 10
2.7 Feature availability matrix . . . 10

3 QTI Reference design **13**
3.1 Sensors on QTI Reference Design . . . 13
3.2 SSC QUP details . . . 14
3.3 Sensor bus connectivity . . . 16
3.4 Sensor interrupt and control GPIOs . . . 16
3.5 Sensor power rails . . . 17

4 Flow diagrams **18**
4.1 Streaming use case . . . 18
4.2 QSH router use case . . . 19
4.3 eNPU sensors use case . . . 19
4.4 RSB use case . . . 20

5 QSH **21**
5.1 Overview . . . 21
5.2 Sensing hubs . . . 25
5.3 Registry . . . 26
5.4 QSH Router . . . 26

6 Application processor **28**
6.1 Source code structure . . . 28
6.2 QSH client APIs . . . 29

7 Sensor Wear Microcontroller **32**

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 0:30:44 GMT
hyungchul.won@lge.com

7.1 Source code structure . . . 32
7.2 QSH APIs . . . 33
7.3 Sample algorithm . . . 33

8 LPAIDSP 34
8.1 Source code structure . . . 34
8.2 QSH APIs . . . 34
8.3 Sample algorithm . . . 34
8.4 Low power island mode . . . 35

9 Bringup/Customization 36
9.1 Introduction . . . 36
9.2 SWM QSH sensor integration . . . 36
9.3 LPAIDSP QSH sensor integration . . . 44
9.4 QSH sensor verification . . . 51
9.5 RSB integration . . . 52
9.6 Sensor axis mapping and calibration . . . 53

10 Memory usage 54
10.1 SWM image memory usage . . . 54
10.2 LPAIDSP image memory usage . . . 54

11 Tools to test sensors 55
11.1 Android applications . . . 55
11.2 Android native SEE test tools . . . 57

12 Sensors vendor ecosystem 59

13 Build and sideloading 60
13.1 Build SWM and LPAIDSP . . . 60
13.2 Sideload SWM and LPAIDSP . . . 60

14 Debug methods 61
14.1 Sensor HAL logs . . . 61
14.2 QXDM Professional . . . 61
14.3 On-device diag logging . . . 62
14.4 Logging over Zephyr shell (UART) . . . 62
14.5 Ramdump . . . 62

15 Sensors customer support 63

16 References 64
16.1 Related documents . . . 64
16.2 Acronyms and terms . . . 64

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

1 Sensors Architecture

The Qualcomm® sensing hub (QSH) functionalities and data paths span across multiple subsystems to efficiently support variety of use cases in the low power mode.

The following diagram depicts a high-level sensors architecture followed by a table that describes each block of the architecture.

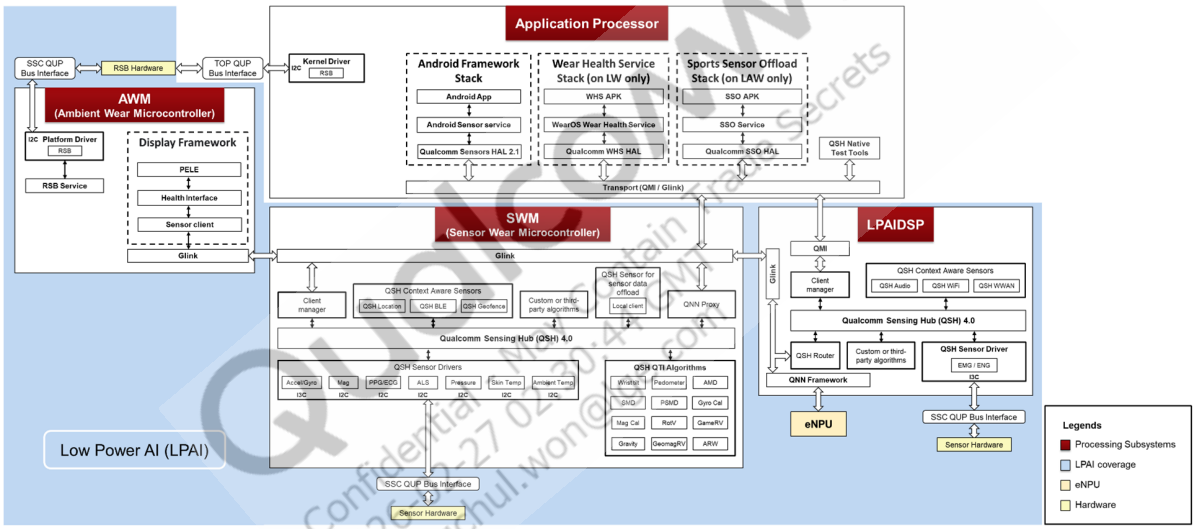


Figure: Sensors architecture

QSH related subsystems

Subsystem	Description
Application Processor	**Android app:** An Android application that utilizes sensor data using the Android framework APIs. **Android sensor service:** Invokes the sensor hardware abstraction layer (HAL) interfaces to provide sensor functionality to the Android framework application/client. **Sensor HAL:** Provides interfaces to the Android sensor service for sensor operations and manages interactions with QSH. **Wear Health Service (WHS) HAL on Linux wearables (on LW only):** Provides interfaces to the WHS service and manages the WHS interactions with QSH. **Sports sensor offload (SSO) HAL on Linux Android wearables (on LAW only):** Provides sensor data offload capability for rendering on the display watchface. **QSH native test tools:** Set of tools to test the sensor functionality. **Rotary side button (RSB) sensor driver:** Provides RSB sensor scroll rotation data in interactive mode.
Low Power Artificial Intelligence (LPAI)	LPAI provides various capabilities and consists of multiple subsystems that operate in lower power mode than the application processor. LPAI is independent to perform operations until there is a need of application processor-based functionality.
Sensor Wear Microcontroller (SWM)	**Physical sensors:** Most physical sensors connects to the SWM. **QTI algorithms:** Algorithms provided by QTI for various activities, such as detect device orientation, activity recognition, and step count. **QSH location:** Provides GPS position fixes and measurements/clock information events. **QSH geofence:** Provides a way to set up a geo boundary using latitude, longitude, and radius to receive events when the device breaches the geofence. **QSH Bluetooth low energy (BLE):** Provides BLE advertising and BLE scanning functionality. **Qualcomm neural network (QNN) proxy:** Provides access to the embedded neuron processor unit (eNPU) hardware via QNN framework on the LPAIDSP. It is useful for the artificial intelligence (AI) use cases.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 07:29:44 GMT
hyungchul.w@qti.com

2 Sensors features

This section describes various sensors related features.

2.1 LPAI processing engines

Module	LPAI modules Capabilities
Processing units	• 2 Cortex M55 microcontroller unit (MCU): SWM and AWM • 1 Qualcomm® Hexagon Processor v79: LPAIDSP
LPAI Global Tightly coupled memory (TCM)	• 3.5 MB shared memory between LPAIDSP, SWM, AWM, and eNPU
Important functionalities	• Sensors and algorithms • Audio • eNPU / AI • Display
Important hardware blocks	• eNPU6 • Low power island (LPI) GPIOs • Snapdragon sensors core Qualcomm Universal Peripherals (SSC QUPs) (14 serial engines, 6 dedicated for sensors.)

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:47 GMT hyungchul.wonchulge.com

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

2.2 Sensing hubs

SWM and LPAIDSP subsystems

Module	Capabilities	
	SWM	**LPAIDSP**
CPU / Maximum frequency	Cortex M55 MCU / 499.2 MHz	Qualcommr Hexagon Processor v79 / 1651.20 MHz
Dedicated/shared	Dedicated for sensors	Shared with audio, other workloads
Hardware thread	1	4
RTOS	Zephyr v3.7.0 RTOS	Qualcomm Real Time (QuRT™) OS
TCM / Island memory	3.5 MB (2 MB ITCM + 1.5 MB DTCM)	2.5 MB island memory
Sensor framework	QSH 4.0	QSH 4.0
Physical sensors	• Accelerometer/Gyroscope • Magnetometer • Pressure • Ambient light sensor (ALS) • PPG/ECG • Skin and ambient temperature	Electroneurography (EMG/ENG)
QSH contextual awareness	• QSH Location • QSH BLE • QSH Geofence	• QSH Audio • QSH WWAN • QSH WiFi
eNPU / AI - sensor data processing	Can access eNPU (via proxy driver on SWM, backend on LPAIDSP)	Can access eNPU (via Qualcomm Neural Network (QNN) framework)
QTI QSH algorithms	• Absolute Motion Detection (AMD) • Significant Motion Detection (SMD) • Persistent Stationary / Motion Detection (PSMD) • Pedometer 3.0 • Tilt-to-wake • Activity Recognition on Wearables (ARW) • Qualcomm Gyroscope Calibration (QGyroCal) • Qualcomm Magnetometer Calibration (QMagCal)	N.A.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:39:44 GMT hyungchul.wan@lge.com

2.3 eNPU

Module	eNPU module
Capabilities	
Version	eNPU6
Processing unit	Dual (Big/Little) eNPU 1024 + 256 MAC
Computational power	Up to 2 TOPS peak (int8)
Key features	• Offload of AI workloads from SWM and LPAIDSP • Programmable via QNN • Does not require any application processor initiation or processing • Support for client priority and core selection/affinity
Typical use cases	• Big eNPU: Compute heavy workloads • Little eNPU: Memory bandwidth heavy workloads and lower complexity • **Examples:** – Sensors – Activity recognition – Voice activation – Noise suppression – Context detection

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

2.4 Ambient WM

Module	AWM subsystem
Capabilities	
CPU / Maximum frequency	Cortex M55 MCU / 499.2 MHz
RTOS	Zephyr v3.7.0 RTOS
TCM	2 MB (1.25 MB ITCM + 0.75 MB DTCM)
Physical sensors	RSB

2.5 Software features

QSH framework

The following table describes key QSH features and capabilities.

QSH features and capabilities

Feature	Capabilities
Framework execution model	Multi-Threaded, can be scaled to single threaded with Core QSH.
Performance	Event driven parallel execution maximizes system performance.
Low power operating mode	Low power Island mode, seamless transition with Normal mode.
Power management	Dynamic power management per use case concurrency.
Memory management	Dynamic memory management support for sensors.
OS dependency	Portable across multiple platforms/OSes.
Dynamic loading	Sensor can be dynamically loaded.
API compatibility	Supported through protobuf messages.
Multi-tech fusion platform	Audio, Location, Bluetooth, WiFi, and WWAN.
New sensor/feature addition	Extensible without change in framework software.
Embedded AI	Integrated with eNPU.
Sensor batching	Multi-player support optimizing system power and performance.
Remote file service	Persistent Sensor configuration, calibration with dynamic update.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungmin.kim@lge.com

Activity recognition

Activity recognition on wearables (ARW) 5.0 is an AI-powered algorithm designed to accurately detect and classify user activities in real time. It supports recognition of key movement patterns including biking, riding in a moving car, running or jogging, walking, and even stationary or fidgeting behavior. This makes it ideal for applications in fitness tracking, mobility analysis, and health monitoring.

User State	Description
BIKE	User on a non-stationary bike
CAR	User in a non-stationary Car (Gasoline Car)
RUN	User running or jogging
STATIONARY	User stationary or fidgeting
WALK	User walking

Sensor data offload

SW6100 platform allows rendering sensor data on the display watchface. The feature is achieved differently depending on the software baseline as described in the following table:

Software product	Modules involved
LW	• WHS • Display offload
LAW	• SSO • Display offload

2.6 Rotary side button

The RSB sensor is handled differently between interactive and ambient modes. In interactive mode, the application processor handles RSB sensor; whereas in ambient mode the AWM handles RSB sensor, which is also called as the RSB offload functionality. This RSB sensor handling is managed through:

- A shared/MUXed QUP interface between SSC_QUP and TOP_QUP over eGPIO.
- A common LDO for RSB to control from application processor and AWM.
- Linux kernel driver for RSB in interactive mode and a proprietary RSB driver for ambient mode.
- RSB offload entry/exit handshake between application processor and AWM.

A high-level depiction of RSB sensor handling in the QTI reference design is covered in RSB use case.

2.7 Feature availability matrix

The following table shares information about availability of key features. These features would likely be enabled at the mentioned milestone with as applicable/known maturity/test level, but the timelines are subject to change.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:14 GMT kwunschul.wun@qne.com

Topic	Feature	Milestone / Level
Features	**Accelerometer/Gyroscope, Magnetometer, Pressure, Skin and ambient temperature, and EMG/ENG**	**ES** / L0
	PPG	Post ES2
	ALS	Post ES2
	RSB	ES2
	All QTI algorithms	FC / L0
	LPAIDSP OEM1 sample algorithm	**ES**
	LPAIDSP qsh_oem1 and ai_oem1 sample algorithms	Post ES2
	LPAIDSP QSH registry sensor	ES2
	SWM OEM1 sample algorithm	ES2
	SWM qsh_oem1 and ai_oem1 sample algorithms	Post ES2
	SWM QSH registry sensor	ES2
	WHS	FC / L0
	SSO	FC / L0
	eNPU sensor use cases	Pre-CS / L0
	Wrist orientation	FC / L0
	QSH Router	Post ES2 / L0
	QSH BLE	Will be updated in future
	QSH Wi-Fi	Pre-CS
	QSH Audio	Post ES2
	QSH Location	ES2
	QSH Geofence	Will be updated in future
	QSH WWAN	**ES**
Test Tools	**Unified Sensor Test Application**	**ES**
	QsensorTest Application	**ES**
	ssc_sensor_info and see_workhorse	**ES**
	ssc_drva_test	ES2 or FC
Debug methods	**SWM Zephyr shell logging**	**ES**
	SWM QXDM and on-device logging	Will be updated in future
	LPAIDSP QXDM and on-device logging	**ES**
	LPAIDSP Ramdump	**ES**

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 12:20:24 GMT hyungkyu.won@lge.com

Topic	Feature	Milestone / Level
	SWM Ramdump	FC

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

3 QTI Reference design

This section provides sensors connectivity details of Qualcomm® SW6100 WDP and WRD platforms.

Note: QTI strongly recommends that the OEMs use bus interface GPIOs, interrupt/control GPIOs, and power rails, same as QTI reference design. If OEMs want to use different configuration, then review and receive approval from Qualcomm Hardware, PMIC, and Sensors teams in advance of designing a hardware platform.

3.1 Sensors on QTI Reference Design

List of sensors on QTI reference platforms

Sensor Type	Part (Vendor)	Interface
Accelerometer/Gyroscope	LSM6DSV32X (ST)	I^3C
Magnetometer	AK09920C (AKM)	I^2C
Ambient light	OPT3004 (TI)	I^2C
Pressure	BMP585 (Bosch)	I^2C
PPG/ECG	MAX86178 (ADI)	I^2C
Skin and Ambient Temperature	CT7117 (SensyLink) - 2 units	I^2C
Electroneurography (EMG/ENG)	STENG1AX (ST) - 4 units	I^3C
Rotary Side Button (RSB) / Optical Track Sensor (OTS)	PAT9401E1 (PixArt)	I^2C

Note: Maximum sample rate support for the streaming sensors will be updated in future revision.

Qualcomm Confidential - May Contain Trade Secrets
2026-03-12 07:14 GMT
hyungchul.kim@lge.com

3.2 SSC QUP details

Note: SSC QUP must be used for the sensor bus connectivity.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

Supported SSC QUP details

MSM GPIO#	SSC GPIO#	SSC QUP SE#	Supported Protocols	MUX'ed with TOP QUP
GPIO_103-104	SSC_0-1	0	I^3C (/I^2C)	N.A.
GPIO_105-106	SSC_2-3	1	I^2C (/I^3C)	N.A.
GPIO_107-110	SSC_4-7	2	UART-4W (/SPI/I^3C/I^2C) SSC_SE2 support 6 lanes SSC_4-9 if needed	N.A.
GPIO_111-112	SSC_8-9	3	I^3C (/I^2C)	Yes, TOP_SE3
GPIO_113-116	SSC_10-13	4	2 I/Os + Debug 2W UART (/SPI/I^2C)	N.A.
GPIO_117-118	SSC_14-15	5	UART-2W (/I^2C)	Yes, TOP_SE6 (L2, L3)
GPIO_121-122	SSC_18-19	7	I^2C (/UART)	N.A.
GPIO_123-124	SSC_20-21	8	I^2C (/I^3C)	N.A.
GPIO_125-126, GPIO_119-120	SSC_22-23, SSC_16-17	9	UART-4W (/I^3C/I^2C) SSC_SE6 MUX'ed with SE9 GPIO_119-120	N.A.
GPIO_72 GPIO_40	SSC_24-25	10	I^2C	Yes, TOP_SE8
GPIO_131-134	SSC_26-29	11	SPI (/I^2C)	Yes, TOP_SE2
GPIO_28-29	SSC_30-31	12	I^2C	Yes, TOP_SE9 (and TOP_SE6 L0, L1)
GPIO_127-130	SSC_32-35	13	SPI (/I^2C)	Yes, TOP_SE7

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

3.3 Sensor bus connectivity

SSC QUP usage for sensors

MSM GPIO#	SSC GPIO#	SSC QUP SE#	TOP QUP SE#	QUP Bus instance	Sensors connected
GPIO_103-104	SSC_0-1	0	N.A.	1	Accelerometer/ Gyroscope
GPIO_105-106	SSC_2-3	1	N.A.	2	ENG-1, ENG-2
GPIO_111-112	SSC_8-9	3	N.A.	4	ENG-3, ENG-4
GPIO_121-122	SSC_18-19	7	N.A.	8	PPG/ECG
GPIO_123-124	SSC_20-21	8	N.A.	9	Magnetometer, Pressure, Ambient light, Skin and ambient temperature
GPIO_72 GPIO_40	SSC_24-25	10	N.A.	11	For RSB in Ambient mode
	N.A.	N.A.	8	N.A.	For RSB in Interactive mode

3.4 Sensor interrupt and control GPIOs

Sensors Control, Clock and Interrupt GPIOs

MSM GPIO#	Used/reserved on QTI reference platforms	Description
GPIO_12	ENG_PWM_OUT	ENG clock
GPIO_24	PPG_INT2	PPG sensor interrupt
GPIO_25	ALSP_INT_N	ALS interrupt
GPIO_26	ECG_FET_CTRL	Control ECG Electrode
GPIO_30	IMU_INT1	Accelerometer/gyroscope interrupt
GPIO_31	PPG_INT1	PPG sensor interrupt
GPIO_98	PRESS_INT	Pressure sensor interrupt*
GPIO_99	RSB_RESET_N	RSB hardware reset pin
GPIO_102	RSB_INT	RSB interrupt

Note: The interrupt GPIO with an asterisk (*) isn't used by default in the SW6100 QTI reference

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungsoo.won@lge.com

design but is reserved for the sensor interrupt purpose and can be utilized if required.

3.5 Sensor power rails

Power rails usage

Sensors Power Rails Usage

Sensor type	Power rails
Acelerometer/Gyroscope	Power rail: L13A (1.8 V) Usage: Connected to VDD/VDDIO of the sensor part
Magnetometer	
Ambient light	
Pressure	
Skin and ambient temperature	
ENG/EMG	
PPG/ECG	Power rail: L13A (1.8V) Usage: Connected to VDD of the sensor part
	Power rail: L22A (4.5V) Usage: Connected to VDD_LED of the sensor part
RSB	Power rail: L13A (1.8V) Usage: Connected to VDD of the sensor part
	Power rail: VREG_BOB (3.6V) Usage: Connected to VLD of the sensor part

Power rail and name mapping for sensor registry

Sensors Power Rails mapping for sensor registry

Power rail	Power rails name for sensor registry	
	SWM	LPAIDSP
L13A	/vrm/l13a	ldoa13
L22A	/vrm/l22a	N.A.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungwoo.lee@lge.com

4 Flow diagrams

This chapter provides high level flow diagrams for various use cases.

4.1 Streaming use case

The following flow diagram depicts high level overview of QSH sensor streaming by the application processor client.

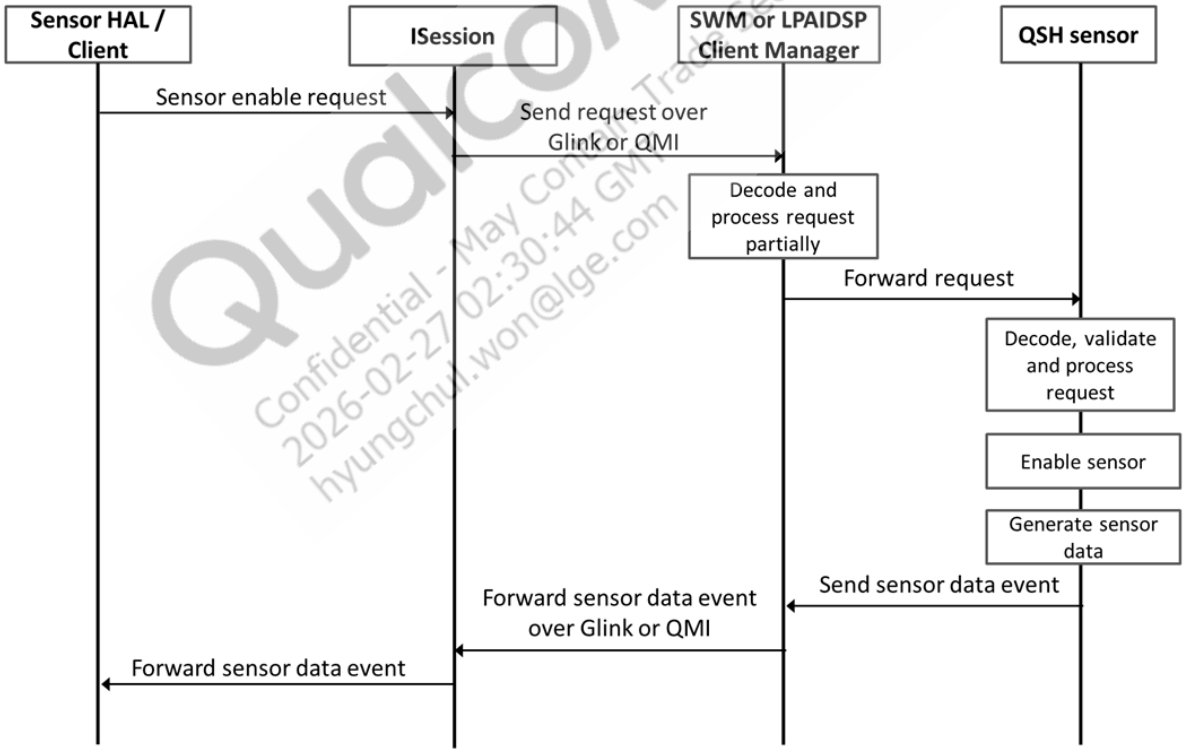


Figure: QSH sensor streaming

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

4.2 QSH router use case

The following flow diagram depicts high level overview of the LPAIDSP sensor receiving SWM QSH sensor data via QSH router.

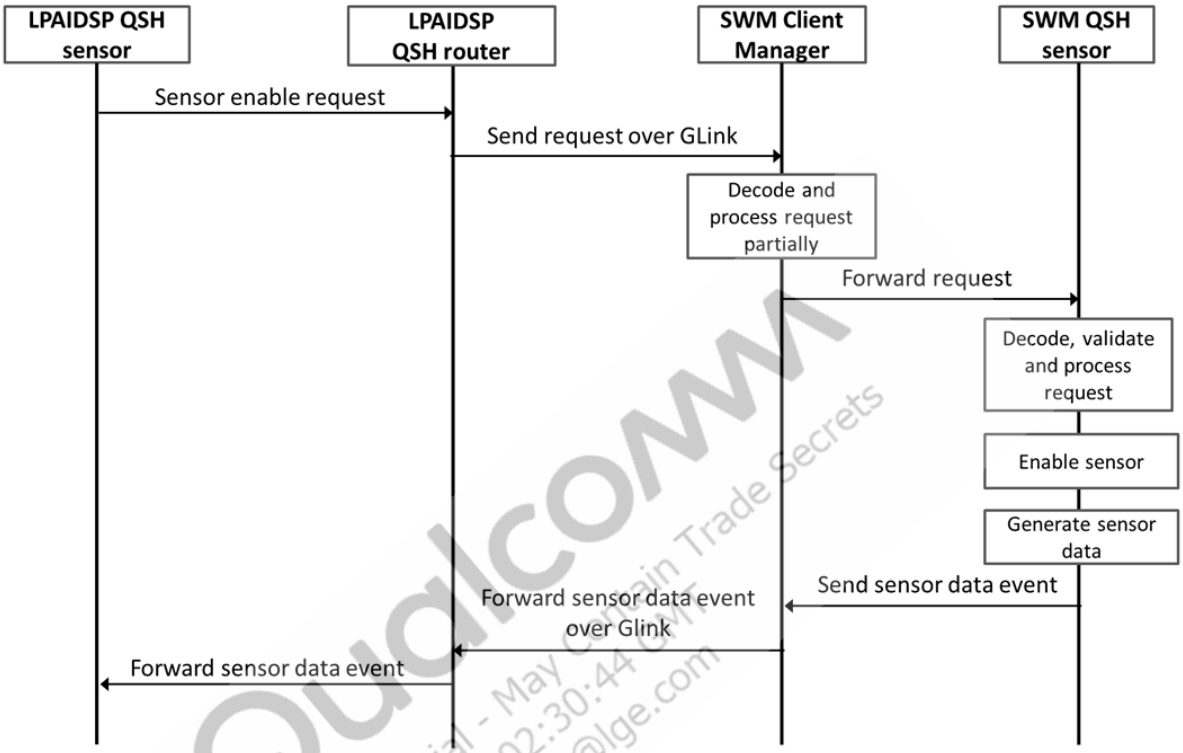


Figure: Receiving data via QSH router

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 02:30:44 GMT hyungchul.won@lge.com

4.3 eNPU sensors use case

The following flow diagram depicts a high level overview of eNPU sensors use case with the LPAIDSP and SWM QSH.

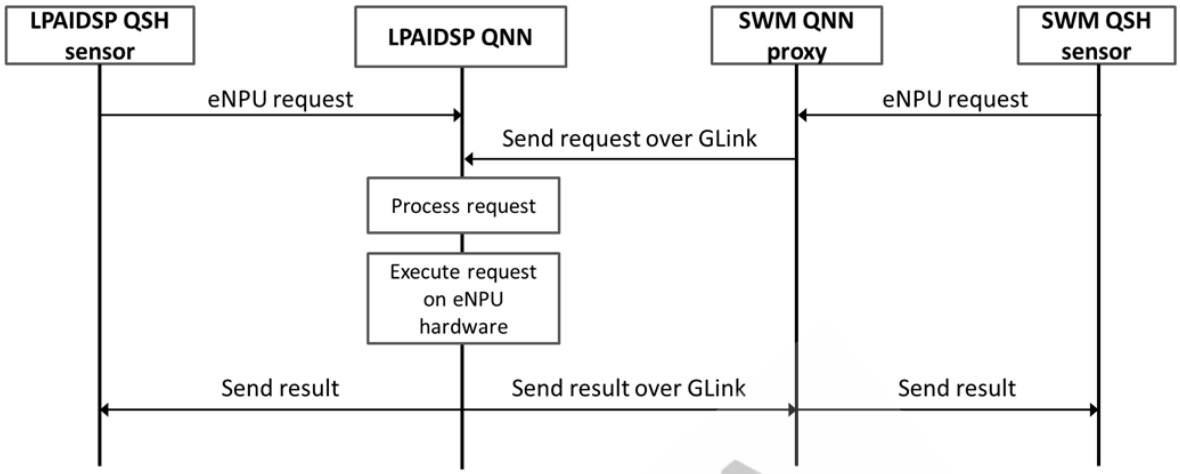


Figure: eNPU sensors use case with SWM and LPAIDSP QSH

4.4 RSB use case

The following flow diagram depicts high level overview of RSB sensor handling in the QTI reference design.

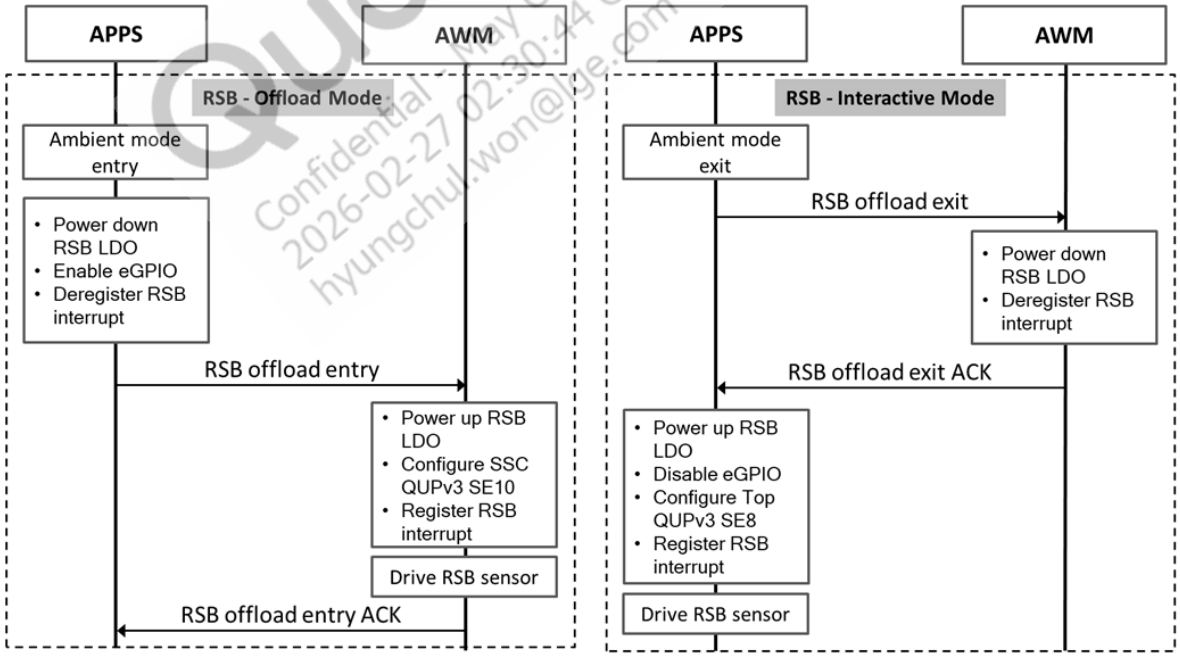


Figure: Handling RSB sensor in reference design

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

5 QSH

This chapter provides QSH overview and functionalities specific to the SW6100 platform.

5.1 Overview

This section introduces the QSH framework. For detailed QSH functionality, see the following topics in Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35).

- Sensors Execution Environment on Page 7-17
- SEE Sensor API on Page 18-25
- SEE Service Manager and Services on Page 34-39
- SEE Platform Sensors on Page 40-46
- Registry in SEE on Page 69-88

The following table describes a few common QSH terminologies.

Qualcomm Confidential May Contain Trade Secrets 2026-02-27 02:00:44 GMT hyungchul.won@lge.com

QSH terminology

Item	Description
Sensor	• Produces a single type of data. For example, accelerometer, gyroscope, timer, interrupt, and rotation vector. • Handles asynchronous data. • Publishes mandatory and custom attributes, and manages its instances.
Sensor instance	• Runs at a specific configuration, publishes output data events, and can be created per client request or shared among multiple requests. • Physical sensors usually share a single instance.
Sensor unique identifier (SUID)	A unique 128-bit ID for each sensor.
Service	A module that provides a synchronous interface for common utilities.
Data stream	A unique connection between a client and data source.
Request	A configuration message that a client sends to a sensor (see `sns_request.h` file).
Event	Asynchronous output data message that a sensor instance generates (see `sns_sensor_event.h` file).
Nanopb	A small code-size protocol buffer implemented in ANSI-C.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:41 GMT hyunchol.woo@lge.com

The QSH framework includes the following components:

- **Application processor software modules**
  - **Client application**: It interacts with the QSH client APIs on the application processor side.
  - **QSH client APIs**: It offers high-level APIs to access services offered by QSH. It simplifies application development by abstracting system complexities and focusing on the application logic.
- **Low-power processor software modules**
  - **Client manager**: It's in-charge of all communications of the low-power processor with the application processor.

The following table describes responsibilities of the client manager.

Function	Description
Translate incoming requests	The client manager takes incoming requests and translates them into a format that the QSH can understand.
Translate outgoing indications	The client manager receives event messages from the QSH and translates these event messages into outgoing indications in a format that's understandable outside the QSH.
Guarantees batching options	If a client specifies certain batching (store/accumulate locally) options, then the client manager ensures that they meet the batching options. The client manager checks that the data is grouped and sent in the same way as the client has specified, ensuring compliance with the criteria.

Sensor and sensor instances

QSH divides the sensor implementation in two logical units: sensor and sensor instance.

- Sensors are producers or consumers, or a combination of producers and consumers of asynchronous data.
  - Each sensor can have one or more sensor instances.
  - Any request to a sensor for data results in the creation of a sensor instance or sharing of an existing sensor instance.
- The sensor creates sensor instances on demand.
  - Sensors manage the lifecycle and configuration of their corresponding instances and sends configuration updates and initial state events to their clients.
  - Each sensor instance operates with a specific client configuration.
  - The sensor instance of a physical sensor programs the sensor hardware to operate at required configuration.
  - Vendors must serve all client requests with a minimal number of sensor instances.
  - The sensor instance generates and sends a stream of data to all the active clients.
- Many sensors can share and configure a single sensor instance - this mode of operation is typical to a combo driver for hardware sensors, such as:
  - Accelerometer and gyroscope
  - Proximity and ambient light

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:20:44 GMT hyungchul.won@lge.com

Communication among sensors

Every algorithm and sensor driver within the QSH framework is called a sensor, with the standard QSH APIs. Information exchange across these sensors is necessary for any real use case.

All communication to, from, and among the sensors is performed through the request and event messages over data streams. The message payloads are defined in the protocol buffer format, using the nanopb generator, encoder, and decoder. The message payload length, message ID, and timestamp (for events) are communicated within the metadata managed by the QSH framework.

The following figure shows the communication between the data client and the data source, using the data stream:

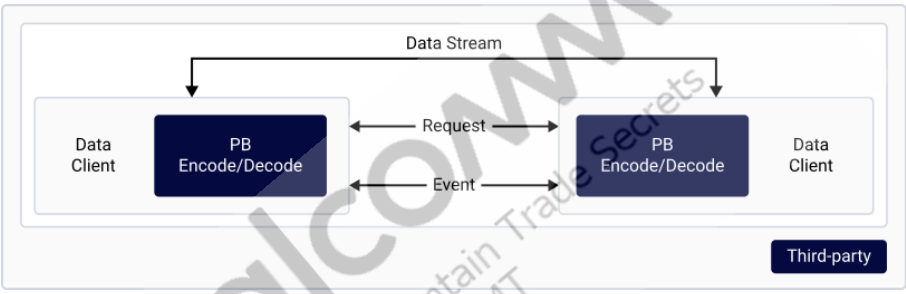


Figure: Sensor communication between client and source

- The client sends request messages to enable, disable, and reconfigure a sensor. Request messages are always addressed to a specific SUID. After the target sensor receives the request message, it sends the request to the sensor instance for proper handling.
- Sensor instances send event messages asynchronously to their registered clients, which can be other sensors or sensor instances.

Qualcomm Confidential - May Contain Trade Secrets 2020-02-27 02:00:44 GMT hyungsul.wan@lm.com

Nanopb protocol buffer in QSH

QSH uses nanopb protocol buffer for the following reasons:

- All the request and event messages that are exchanged between the sensors.
  - A sensor or sensor instance must encode the payload (if present) for all requests it sends to its dependents.
  - A sensor or sensor instance must decode the payload (if present) for all requests it receives.
  - A sensor or sensor instance must encode the payload (if present) for all events it publishes.
  - A sensor or sensor instance must decode the payload (if present) for all events it receives from its dependents.
- Representing the attribute data
  - All attribute values are in the nanopb-encoded format.
- Diagnostic log packet payload
  - All payloads in the diagnostic log packets are in the nanopb-encoded format.

5.2 Sensing hubs

The presence of two subsystems with unique QSH capabilities requires a unique identification to communicate and utilize the functionalities. As mentioned in the following table, the communication mechanism for each sensing hub is different.

QSH sensing hubs

Sensing hub	Sensing hub unique ID	Communication mechanism
SWM QSH	Hub 2	Glink
LPAIDSP QSH	Hub 1	QMI

The following figure depicts a high-level sensing hub capability discovery and request routing, followed by a brief description.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 20:44 GMT zheng.hui.fan@qualcomm.com

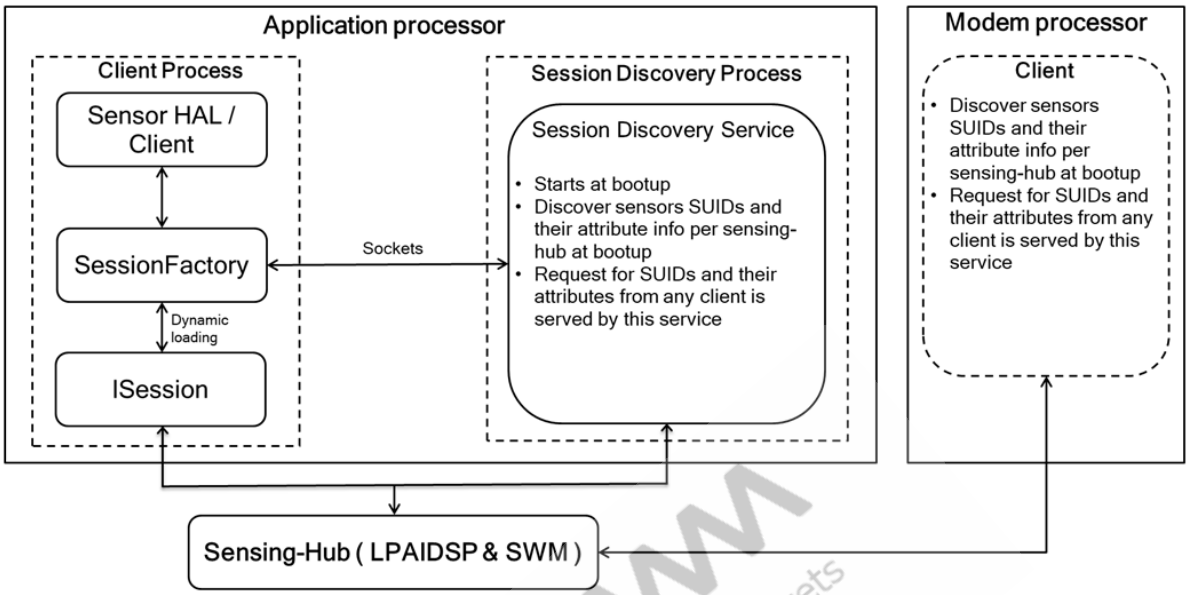


Figure: Sensing hubs

- Sensor HAL and ISession native client on the application processor are Sensing hub aware implementations and automatically uses the required communication mechanism.
- The Android framework and Android apps are not and need not be Sensing hub aware, since the Sensor HAL is responsible to route the client request to respective hub.
- For a non-application processor client such as on modem processor, must use the communication mechanism required for the sensing hub.
- In the beginning, the client discovers the sensors/SUIDs available on the sensing hub and the sensor attributes. Later, when desired, the client and enable or disable the required sensor.

5.3 Registry

The registry contains the platform specific and sensor specific configuration. A separate registry configuration is maintained for each sensing hub.

More details will be provided after the software support is enabled for registry functionality.

5.4 QSH Router

Algorithms integrated in the LPAIDSP QSH, depends on the physical sensor data or algorithm data from the SWM QSH. To facilitate this, a special sensor named QSH router is available in the LPAIDSP QSH.

For the algorithm integrated in the LPAIDSP QSH, the QSH router is transparent and doesn't require any special configuration.

QSH router classifies the QSH sensor as:

- Local sensor, which is local to the sensing hub.
- Remote sensor, which is available on another sensing hub.

Note: QSH router enables only one way access, that is, SWM QSH sensors (sensor drivers, algorithms, and context aware sensors) are accessible by the LPAIDSP QSH algorithms. Whereas, the SWM QSH can't access the LPAIDSP QSH sensors.

The following figure provides an overview of the QSH router design.

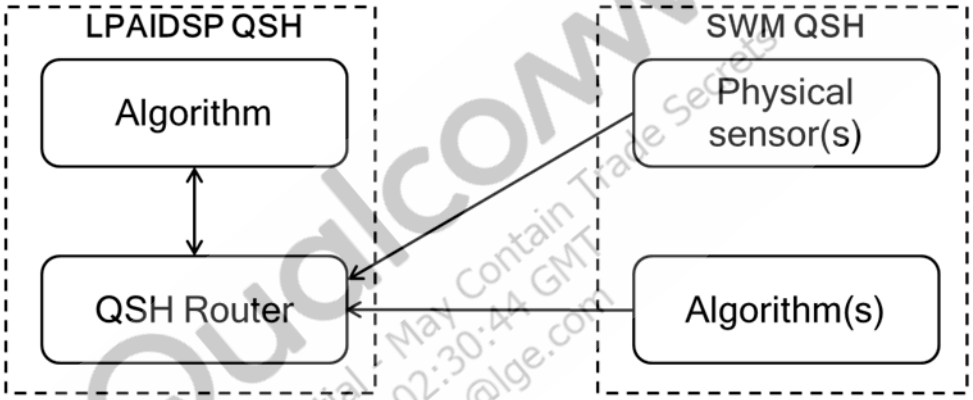


Figure: QSH router

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

6 Application processor

This chapter provides sensors related details on the application processor side.

6.1 Source code structure

Sensors related code on the applications processor side is located at the following locations:

- `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship`

```
vendor/qcom/proprietary/sensors-ship/
├─ api - QSH API component for all public proto APIs
├─ build_config - Build config directory
├─ config - Registry JSON config
├─ core - QSH ISession core
├─ lookup - QSH ISession SUID lookup
├─ services - sscrpcd Service
├─ test - QSH native test apps
└─ utils - QSH utils
```

- `<hlos_workspace>/vendor/qcom/proprietary/sensors-android`

```
vendor/qcom/proprietary/sensors-android/
├─ build_config - Build config directory
├─ sensors-cal-service - Sensor calibrate module
├─ sensors-hal-2.X - QSH sensors HAL
└─ test - Test application: USTA, QSTA
```

- `<hlos_workspace>/vendor/qcom/opensource/sensing-hub`

```
vendor/qcom/opensource/sensing-hub/
├─ apis - QSH API component for all public proto APIs
├─ build_config - Build config directory
├─ sensordaemon/init.vendor.sensors.rc - Sets persist registry folder
permissions
├─ common - Common header
```

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 02:30:44 GMT hyungshul.won@lge.com

```
├─ examples - QSH client API sample application
└─ session - QSH client (ISession) interface
```

6.2 QSH client APIs

QSH APIs

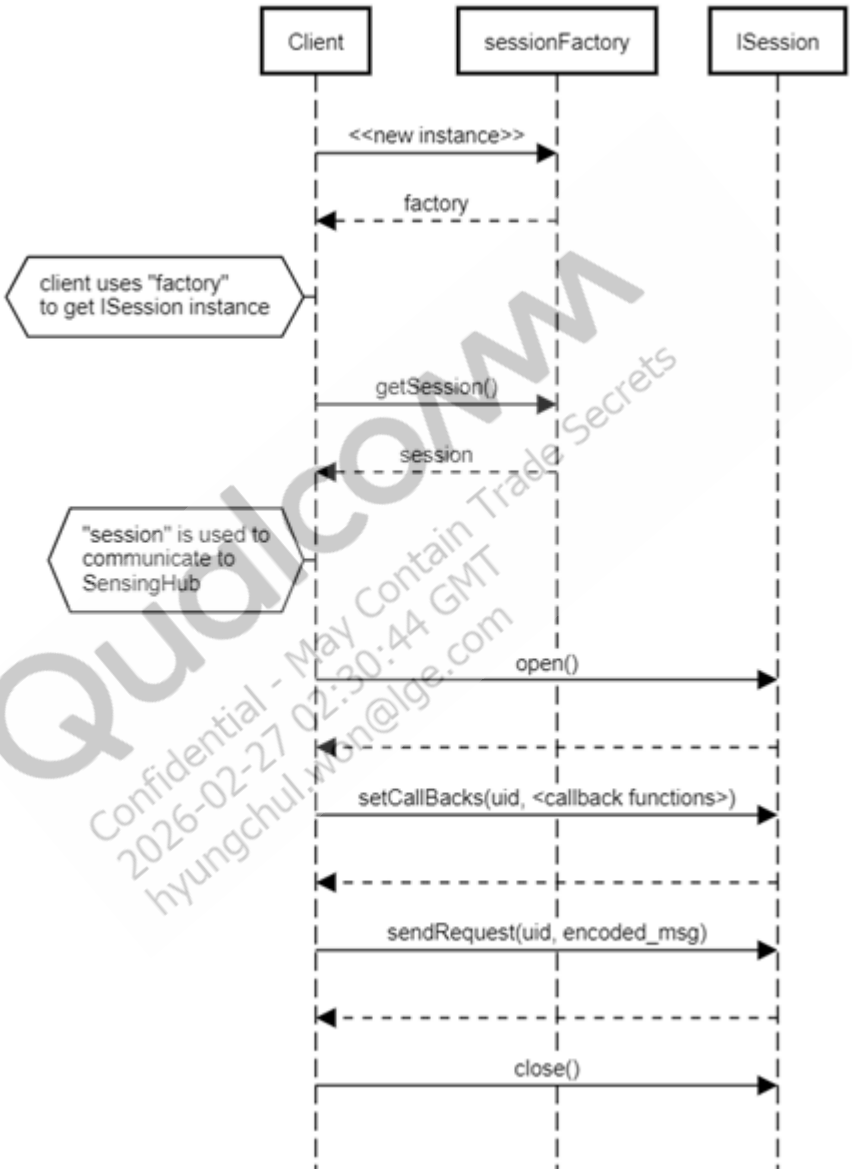
The client must use the ISession APIs provided by QTI, to communicate with sensing hub to enable/disable sensor.

Isession APIs are available in the `<hlos_workspace>/vendor/qcom/opensource/sensing-hub/session/1.0/inc/` directory.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

QSH client API workflow

The QSH-exposed APIs are called QSH client APIs for applications. The client applications interact with the QSH framework available on the low-power processor. The following figure shows the detailed breakdown and usage of the QSH client APIs:



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

Figure: QSH client API

The following list describes the QSH client APIs:

getSession

To interact with the QSH, the sensor application client must create an interface session by invoking the `getSession()` API as shown in the figure. Successful creation of the interface session returns an `ISession*` object to the client.

Open, Close, SetCallback, and SendReques

The interface created using the `getSession()` API operates on a *once_per_session* basis. This interface allows the application to use the QSH client APIs, such as `open()`, `close()`, `setCallback()`, and `sendRequest()`, until the session is explicitly deleted using the `close()` and `delete()` APIs.

After the interface session establishes, the application or the client code can open a sensor session by calling the `open()` API. Similar to `getSession()`, the `open()` API is also invoked on a *once_per_session* basis. Additionally, the `setCallback()` and `sendRequest()` APIs are invoked once for a specified SUID.

Sample client

The `SessionClient` sample application shows how to use QSH client API to develop applications. This sample application builds by default in the `/usr/bin` directory on the device.

For complete sample code, see the `<hlos_workspace>/vendor/qcom/opensource/sensing-hub/examples/SessionClient/SessionClient.cpp` file.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:39:44 GMT hyungchul.wonc@qc.com

7 Sensor Wear Microcontroller

This chapter provides sensors related details on the SWM side.

7.1 Source code structure

`LPAI.WM.1.1-00xxx-LPAI_WM_PROD-y`, contains the SWM source code, where `xxx-y` represents a build number.

The following list shows the sensor related folders and its usage in SWM source:

- `wm_proc/sensinghub`

```
wm_proc/sensinghub
├─ api - QSH API component for all public proto APIs
├─ config - sensing hub framework configuration
├─ drivers - QSH Context Aware Sensors - Location, BLE, Geofence
├─ platform - QSH Core framework, utilities, platform sensors
├─ sensors - QTI reference design sensor drivers
└─ zephyr - Build sensing hub as zephyr module
```

- `wm_proc/sensinghub/sensors`

```
wm_proc/sensinghub/sensors
├─ algorithms - QTI algorithm sensors
└─ drivers - QTI reference design physical drivers
```

- `wm_proc/sensinghub/driver`

```
wm_proc/sensinghub/sensors/drivers
├─ ak0991x - QTI reference design magnetometer sensor driver
├─ bmp5 - QTI reference design pressure sensor driver
├─ build - Build directory
├─ cmake_build - zephyr cmake build directory
├─ ct7117x - QTI reference design skin and ambient light sensor
driver
└─ lsm6dsv - QTI reference design accelerometer/gyroscope sensor
driver
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.wonq@qc.com

7.2 QSH APIs

QSH APIs includes the following:

- QSH sensor APIs: `wm_proc/sensinghub/api/pb and wm_proc/sensinghub/platform/api`
- QSH service APIs: `wm_proc/sensinghub/platform/inc/services`
- QSH context aware sensor APIs: `wm_proc/sensinghub/drivers`

7.3 Sample algorithm

Sample algorithms in SWM QSH will be enabled in future software release. Until then, the sample algorithms on LPAIDSP QSH can be used for testing. See the QSH APIs mentioned in QSH APIs and the LPAIDSP sample algorithms mentioned in Sample algorithm. To port the algorithm into SWM QSH:

1. Copy the LPAIDSP algorithm folder into SWM QSH.
2. See Convert SCons to CMake to convert SCons build script into CMake script.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

8 LPAIDSP

This chapter provides sensors related details on the LPAIDSP side.

8.1 Source code structure

`LPAIDSP.HT.1.4-00xxx-ASPEN-y`, contains the LPAIDSP source code, where `xxx-y` represents a build number.

The following list shows the sensor related directories and its usage in the LPAIDSP source.

```
adsp_proc/
├─ qsh_algorithms - QSH - All fusion algorithms
├─ qsh_api - QSH API component for all public proto APIs
├─ qsh_drivers - QSH Context Aware Sensors - WWAN, WiFi, Audio
├─ qsh_platform - QSH Core framework, utilities, platform sensors
└─ ssc_drivers - QTI reference design physical sensor drivers
```

8.2 QSH APIs

QSH APIs includes the following:

- QSH sensor APIs: `adsp_proc/qsh_api/pb and adsp_proc/qsh_platform/api`
- QSH service APIs: `adsp_proc/qsh_platform/inc/services`
- QSH context aware sensor APIs: `adsp_proc/qsh_drivers`

8.3 Sample algorithm

Reference algorithms, `oem1`, `qsh_oem1`, and `ai_oem1` are available to explore how the QSH APIs are used, and algorithms can request data from physical sensor or another algorithm.

- `oem1` algorithm: Demonstrates adding dependency of a physical sensor, sending request to physical sensors, processing received physical sensor data, sending request to registry

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

sensor to read the registry items and processing registry event.

- `qsh_oem1` algorithm: Demonstrates adding dependency of other algorithms, sending request to algorithms, and processing received algorithm data.
- `ai_oem1` algorithm: Demonstrates sensor data processing on eNPU via QNN framework.

The reference algorithms are located at `adsp_proc/qsh_algorithms`.

To enable the sample algorithm, add the SCons filename of the algorithm (without SCons extension) to `include_qsh_algorithms_libs.extend` in the `adsp_proc/qsh_platform/chipset/aspen/por.py` file.

See QSH sensor verification, to verify the sample algorithm.

8.4 Low power island mode

The LPAIDSP subsystem supports a low power execution mode called Low Power Island (LPI) mode.

While LPAIDSP is in LPI mode:

- Island code is a subset of the entire image.
- LPAIDSP is self-sufficient to execute only from L2/TCM memory, with reduced - island only functionality.
- During Island mode of operation, code and data needed for operation are copied from DDR to L2/TCM.
- DDR cannot be accessed, but could access other internal memories to LPAI.
- LPAIDSP runs in a power optimized way and hence at reduced speeds.
- Exhibits reduced power consumption.
- System level low power modes such as XO shutdown and power collapse are supported, where DDR goes into self-refresh mode and other internal buses on SoC required to access non-island modules are turned off.

While LPAIDSP is in Normal mode:

- LPAIDSP executes out of DDR memory, with all the functionalities.
- In Normal mode, LPAIDSP has access to all external memories, like DDR and other chip resources. This mode is used when interactions outside of LPAIDSP are required. Here, performance-intensive operations are possible; LPAIDSP can run at full speed and vote for on-chip resources.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.wan@lge.com

9 Bringup/Customization

This chapter describes bringup and customization steps for QSH sensors on SWM and LPAIDSP.

9.1 Introduction

The sensor customization and bringup typically comprises of the following activities:

- QSH driver/algorithm customization:

  Based on the customization requirement refer the following applicable topics:

  – Customize QTI reference design sensor driver

  – Integrate third-party sensor driver

  – Integrate vendor provided algorithm

  – Develop algorithm

- Sensor axis mapping

  – See Sensor axis mapping and calibration, for more information.

- Sensor calibration

  – Pre-requisites

    ○ Bringup of all physical sensors is complete

    ○ Axis mapping is correct for all the sensors

  – See Sensor axis mapping and calibration, for more information.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.wan@lge.com

9.2 SWM QSH sensor integration

This section covers customization steps for SWM.

QSH sensor typically uses SCons for building, but since the SWM is based on Zephyr OS, the build system needs to be converted from SCons to CMake to compile the related files. See Convert SCons to CMake, for detailed steps.

Note: Island mode is not applicable to the SWM. QSH drivers and algorithms are developed with the Island feature support that is available on LPAIDSP and hence contain `_island` filename suffix. If there is no Island support on the subsystem, all `_island` suffix files are compiled in the same way as non-Island files, and this is automatically taken care by build configuration. No change required in filename or compilation steps.

Customize QTI reference design sensor driver configuration

This section is applicable only if the sensor part used is same as the SW6100 QTI reference design but the sensor GPIOs, interrupt pin, or bus type used are different. See the *Registry in SEE* section on page 69-88 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35) while performing the following steps.

SWM side changes

1. The Registry sensor will be enabled in the future software release. Hence, the Registry dependency is disabled in the sensor driver and then the driver uses hardcoded registry configuration.

   The following table shows the location of hardcoded configuration in the driver. Additionally, a few macro values are defined in the `wm_proc/sensinghub/platform/chipset/aspen/see/sns_interface_defs.h` file.

Driver	Hardcoded configuration location
lsm6dsv	lsm6dsv/src/core/sns_lsm6dsv_registry_default_config.c
ak0991x	ak0991x/src/sns_ak0991x_sensor.c
bmp5	bmp5/src/sns_bmp5_registry_default_config.c
ct7117x	ct7117x/src/sns_ct7117x_config.h

2. If the bus interface GPIOs or bus interface type used for sensor is different than it is used for QTI reference design sensor, then perform the following steps to configure the QUP instance as required.

   a. See SW6100 Device Interfaces Guide (80-74841-87) and check whether the desired bus interface type is supported by the SSC QUP SE# serial engine.

   b. Follow the same above document to modify the required QUP instance and/or serial engine configuration as required.

   Note: For any support required for the QUP instance and/or serial engine configuration, raise a Salesforce case with Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral*

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:47 GMT hyunchan.no@lge.com

Buses(UART/I2C/SPI/TSIF/etc) - non AP.

c. If the bus type is being changed for SSC QUP SE#0 from default I^3C to any other bus type, then comment out `CONFIG_QC_SNS_I3C_DEVICE_INIT_PRIO` macro in the `wm_proc/sensinghub/config/prj.conf` file.

d. Use the QUP instance ID as `bus_instance` registry item value of the driver.

e. Set `bus_type` item value to match the bus interface type.

f. Set `slave_config` item value with *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for SPI bus interface.

g. Set the `min_bus_speed_khz` item with minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.

h. Set `max_bus_speed_khz` to match supported maximum bus speed as per sensor data sheet and the maximum speed supported by bus interface of QUP serial engine.

   i. To know QUP serial engine's maximum frequency for bus interface type, see SW6100 Device Interfaces Guide (80-74841-87).

   ii. To know sensor hardware maximum bus interface frequency support, refer the Sensor Data Sheet.

   For example (only for reference, see above mentioned documents to confirm) - Typically, the following QUP serial engine's maximum frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

   Typically, the following sensor hardware's maximum bus interface frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

   Based on this example, following is the appropriate value for `max_bus_speed_khz` item:

Qualcomm Confidential - May Contain Trade Secrets 2020-02-27 02:30:44 GMT hyungchul.won@nxp.com

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

i. If I^2C or SPI interface is used for the sensor, then the set driver registry configuration item `dri_irq_num` with the GPIO number is used for the interrupt.

3. After compiling the SWM code, verify that the driver registration function (see `register_func_name` value in driver scons file to know the registration function name) is added to `sns_static_drivers_list[]` in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/drivers/sns_static_drivers.c` file.

Verify sensor

Refer QSH sensor verification to verify sensors.

Integrate third-party sensor driver

Perform the following steps to integrate third party sensor driver on the SWM. See the *Registry in SEE* section on page 69-88 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35) while performing the following steps.

SWM side changes

1. Acquire the directory from vendor that contains the driver, associated SCons build file, JSON configuration files for driver, and any custom protobuf file for driver.
2. Place the acquired directory in `wm_proc/sensinghub/sensors/drivers`.
3. If the driver has custom protobuf file (.proto), copy the file to `wm_proc/sensinghub/api/pb`.
4. To enable the sensor driver for compilation, add driver SCons filename (without the SCons extension) into `include_sensor_vendor_libs` array in the `wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake` file.
5. See Convert SCons to CMake, to create `CMakeLists.txt` under the build folder for compiling.
6. Add the driver directory to compile using the `add_subdirectory()` function in the `wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake` file.
7. The Registry sensor will be enabled in future software release. Hence, in the *Registry* dependency must be disabled in the driver.

Ensure that the driver Scons file handles `SNS_DISABLE_REGISTRY` macro to disable the registry operations and then in the driver hardcode the values matching to the JSON config file. If the driver Scons file doesn't support `SNS_DISABLE_REGISTRY` macro, then receive a modified version of

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:00:44 GMT hyungchul.won@qc.com

the driver without registry operations and with hardcoded values in the driver matching to the JSON config file.

For example, the LSM6DSV driver has hardcoded configuration defined in the `sns_lsm6dsv_registry_def_config()` function. Similarly, the other sensor driver would have a dedicated function.

8. If the bus interface GPIOs or bus interface type used for sensor is different than it is used for QTI reference design sensor, then perform the following steps to configure the QUP instance as required.

   a. See SW6100 Device Interfaces Guide (80-74841-87) to check whether the desired bus interface type is supported by the SSC QUP SE# serial engine.

   b. Follow the same above document to modify the required QUP instance and/or serial engine configuration.

Note: For any support required for the QUP instance and/or serial engine configuration, raise a Salesforce case with the Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

---

   c. If the bus type being used SSC QUP SE#0 as I^3C, then replace the `lsm6dso0` keyword in teh `wm_proc/sensinghub/platform/chipset/aspen/see/sensinghub.overlay` file and also within `SENSOR_DEVICE_DT_DEFINE(DT_NODELABEL(lsm6dso0),...` line in the `wm_proc/sensinghub/platform/pal/i3c/zephyr/sns_com_port_i3c.c` file with a suitable keyword for the third-party driver.

   d. If the bus type is changed for SSC QUP SE#0 from default I3C to any other bus type, then comment out `CONFIG_QC_SNS_I3C_DEVICE_INIT_PRIO` macro in the `wm_proc/sensinghub/config/prj.conf` file.

   e. Use the QUP instance ID as `bus_instance` registry item value of the driver.

   f. Set `bus_type` item value to match the bus interface type.

   g. Set `slave_config` item value with *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for SPI bus interface.

   h. Set the `min_bus_speed_khz` item with minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.

   i. Set `max_bus_speed_khz` to match supported maximum bus speed as per the Sensor Data Sheet and the maximum speed supported by bus interface of the QUP serial engine.

   i. To know the QUP serial engine's maximum frequency for bus interface type, see SW6100 Device Interfaces Guide (80-74841-87).

ii. To know the sensor hardware maximum bus interface frequency support, refer the Sensor Data Sheet.

For example (only for reference, see above mentioned documents to confirm) - Typically, the following QUP serial engine's maximum frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

Typically, the following sensor hardware's maximum bus interface frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

Based on this example, following is the appropriate value for `max_bus_speed_khz` item:

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

9. If I^2C or SPI is interface used for the sensor, then the set driver registry configuration item `dri_irq_num` with the GPIO number is used for the interrupt.

10. Modify `num_rail`, `vdd_rail`, and `vddio_rail` item values according to the power rail(s) required for the sensor.

    a. Set `num_rail` item value to denote the total number of power rail(s) required for the sensor.

    b. Refer power-rail-and-name-mapping and note the power rail name to be used in the sensor registry.

    c. Based on the power rail(s) used for sensor, keep only the required `vdd_rail` and/or `vddio_rail` item in the configuration. For example, if the sensor uses a single power rail, then the `set num_rail` item value to `1` and keep only the applicable `vdd_rail` or `vddio_rail` item in the configuration.

12. After compiling the SWM code, verify that the driver registration function (see `register_func_name` value in the driver scons file to know the registration function name) is added to

Qualcomm Confidential - May Contain Trade Secrets 2023-02-27 02:23:41 GMT hyungchul.wooldge.com

`sns_static_drivers_list[]` in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/drivers/sns_static_drivers.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that is different than the QTI reference design sensor types and if the sensor needs to be exposed to Sensor HAL, Android framework, and Android App then:

a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.

b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then refer the existing `electro_neuro_graph.cpp` or `ambient_light.cpp` file and create a new CPP file to support new sensor type.

2. If the driver has custom protobuf file (.proto), copy the file to `<hlos_workspace>/vendor/qcom/opensource/sensing-hub/apis/proto`.

Verify sensor

Refer QSH sensor verification to verify the sensor.

Convert SCons to CMake

SWM software supports CMake based compilation. Hence, if the third-party QSH driver/algorithm has SCons build file but not the `CMakeLists.txt` file, then it is required to convert the SCons file into the new `CMakeLIsts.txt` file.

1. As an example, refer the existing QTI reference design driver/algorithm `CMakeLists.txt` file to understand how the macros, `if()..else()...endif()` conditions, and others are written.

2. Create a new `CMakeLists.txt` file under the third-party driver/algorithm build directory.

3. Convert the third-party driver/algorithm SCons into the `CMakeLists.txt` file. While porting the changes, you can refer the official documentation Writing CMakeLists Files, for the `CMakeLists.txt` file syntax.

4. Replace the `env.AddSSCSU` function with `add_ssc_su`.

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:30:44 GMT hyungchul.won@qc.com

Integrate third-party provided algorithm

To integrate the third-party sensor algorithm on the SWM:

SWM side changes

1. Acquire the directory from vendor that contains the algorithm, associated SCons build file, JSON configuration files, and any custom protobuf file for the algorithm.
2. Place the acquired directory in `wm_proc/sensinghub/sensors/algorithms`.
3. If the driver has custom protobuf file (.proto), copy the file to `wm_proc/sensinghub/api/pb`.
4. To enable the sensor algorithm for compilation, add the algorithm SCons filename (without the SCons extension) into `include_ssc_algorithms` array in the `ssc_algorithms.cmake` file.
5. See Convert SCons to CMake, to create `CMakeLists.txt` under the build folder for compiling.
6. Add the algorithm directory to compile using the `add_subdirectory()` function in the `wm_proc/sensinghub/sensors/algorithms/cmake_build/ssc_algorithms.cmake` file.
7. If the algorithm has any JSON config file:

The Registry sensor will be enabled in future software release. Hence, the *Registry* sensor dependency must be disabled in the algorithm.

Ensure that the driver Scons file handles `SNS_DISABLE_REGISTRY` macro to disable the registry operations and then in the algorithm hardcode the values matching to the JSON config file. If the algorithm Scons file doesn't support `SNS_DISABLE_REGISTRY` macro, then receive a modified version of the algorithm without registry operations and with hardcoded values in the driver matching to the JSON config file.

8. After compiling the SWM code, verify that the algorithm registration function (see `register_func_name` value in the algorithm scons file to know the registration function name) is added to `ssc_static_algorithms_list[]` in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/algorithms/ssc_static_algorithms.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that is different than the QTI reference design sensor types and if the sensor needs to be exposed to the Sensor HAL, Android framework, and Android App then:

a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-23 08:30:44 GMT hyungchun.yoo@qualcomm.com

b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then refer the existing `gravity.cpp` or `step_count.cpp` file and create a new CPP file to support the new sensor type.

2. If the algorithm has custom protobuf file (.proto), copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto`.

Verify sensor

Refer QSH sensor verification to verify the sensor.

Develop algorithm

Sample algorithms in the SWM QSH will be enabled in future software release. Until then, sample algorithms on the LPAIDSP QSH can be used for testing.

See the QSH APIs mentioned in QSH APIs and the sample algorithms mentioned in Sample algorithm to develop a new algorithm. Additionally, see Convert SCons to CMake to convert SCons build script into CMake script.

9.3 LPAIDSP QSH sensor integration

This section covers customization steps for LPAIDSP.

Customize QTI reference design sensor

This section is applicable only if the EMG/ENG sensor part used is same as the SW6100 QTI reference design but the sensor GPIOs, interrupt pin, or bus type used are different. See the *Registry in SEE* section on page 69-88 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35) while performing the following steps.

LPAIDSP side changes

1. The STENG1AX sensor driver is disabled by default and can be enabled if required. To enable this driver, apply the following code change in `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.

```
-include_sensor_vendor_libs.extend(['sns_da_test'])
+include_sensor_vendor_libs.extend(['sns_da_test', 'steng1ax'])
```

2. The Registry sensor will be enabled in the future software release. The Registry dependency is disabled in the sensor driver and then the driver uses hardcoded registry configuration.

   The registry configuration is hardcoded in the `sns_steng1ax_registry_default_config.c` file of the STENG1AX driver with a few macro values defined in the `adsp_proc/qsh_platform/chipset/aspen/qsh/sns_interface_defs.h` file.

3. If the GPIOs or bus interface type used for sensor is different than the QTI reference design sensor, then follow the below steps to configure the QUP instance as required.

   a. See SW6100 Device Interfaces Guide (80-74841-87) and check whether the desired bus interface type is supported by the SSC QUP SE# serial engine.

   b. Follow the same above document to modify the required QUP instance and/or serial engine configuration as required.

   Note: For any support required for the QUP instance and/or serial engine configuration, raise a Salesforce case with Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

   c. Use the QUP instance ID as `bus_instance` registry item value of the driver.

   d. Set `bus_type` item value to match the bus interface type.

   e. Set `slave_config` item value with *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for SPI bus interface.

   f. Set the `min_bus_speed_khz` item with minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.

   g. Set `max_bus_speed_khz` to match supported maximum bus speed as per the Sensor Data Sheet and the maximum speed supported by bus interface of QUP serial engine.

      i. To know QUP serial engine's maximum frequency for bus interface type, see SW6100 Device Interfaces Guide (80-74841-87).

      ii. To know sensor hardware maximum bus interface frequency support, refer the Sensor Data Sheet.

      For example (only for reference, see above mentioned documents to confirm) - Typically, the following QUP serial engine's maximum frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

      Typically, the following sensor hardware's maximum bus interface frequency is supported:

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:50:34 GMT hunchchen@lge.com

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

Based on this example, following is the appropriate value for `max_bus_speed_khz` item:

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

h. If I^2C or SPI interface is used for the sensor, then the set driver registry configuration item `dri_irq_num` with the GPIO number is used for the interrupt.

4. After compiling the LPAIDSP code, verify that the driver registration function (see `register_func_name` value in driver scons file to know the registration function name) is added to `sns_static_drivers_list[]` in the `adsp_proc/qsh_platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/sns_static_drivers.c` file.

Verify sensor

Refer QSH sensor verification to verify sensors.

Integrate third-party sensor driver

Perform the following steps to integrate third party sensor drivers on the LPAIDSP. See the *Registry in SEE* section on page 69-88 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35) while performing the following steps.

LPAIDSP side changes

1. Acquire the directory from vendor that contains the driver, associated SCons build file, JSON configuration files for driver, and any custom protobuf file for driver.
2. Place the acquired directory in `adsp_proc/ssc_drivers`.
3. If the driver has custom protobuf file (.proto), copy the file to `adsp_proc/qsh_api/pb`.
4. To enable the sensor driver for compilation, add driver SCons filename (without the SCons extension) into `include_sensor_vendor_libs.extend` array in the `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.
5. To enable the Island support for the driver, refer the driver Scons file to find a flag with the `SNS_ISLAND_INCLUDE_` keyword. Then add the rule to enable the same flag into `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.

Qualcomm Confidential - May Contain Trade Secrets
Reader 02-27 02:30:44 GMT
hyunchul.wo@lge.com

For example: `env.AddUsesFlags(['SNS_ISLAND_INCLUDE_XXXX'])`

6. The Registry sensor will be enabled in future software release. Hence, in the *Registry* dependency must be disabled in the driver.

   Ensure that the driver Scons file handles the `SNS_DISABLE_REGISTRY` macro to disable the registry operations and then in the driver hardcode the values matching to the JSON config file. If the driver Scons file doesn't support the `SNS_DISABLE_REGISTRY` macro then receive a modified version of the driver without the registry operations and with the hardcoded values in the driver, matching to the JSON config file.

   For example, the STENG1AX driver has configuration defined in the `sns_steng1ax_registry_def_config()` function. Similarly, the other sensor driver would have a dedicated function.

7. If the bus interface GPIOs or bus interface type used for sensor is different than the QTI reference design sensor, then perform the following steps to configure the QUP instance as required.

   a. See SW6100 Device Interfaces Guide (80-74841-87) to check whether the desired bus interface type is supported by the SSC QUP SE# serial engine.

   b. Follow the same above document to modify the required QUP instance and/or serial engine configuration.

Note: For any support required for the QUP instance and/or serial engine configuration, raise a Salesforce case with the Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

---

   c. Use the QUP instance ID as `bus_instance` registry item value of the driver.

   d. Set `bus_type` item value to match the bus interface type.

   e. Set `slave_config` item value with *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for SPI bus interface.

   f. Set the `min_bus_speed_khz` item with minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.

   g. Set `max_bus_speed_khz` to match supported maximum bus speed as per sensor data sheet and the maximum speed supported by bus interface of QUP serial engine.

   i. To know the QUP serial engine's maximum frequency for bus interface type, see SW6100 Device Interfaces Guide (80-74841-87).

   ii. To know the sensor hardware maximum bus interface frequency support, refer the

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 20:20:44 GMT hyungchul.wonghee.com

Sensor Data Sheet.

For example (only for reference, see above mentioned documents to confirm) - Typically, the following QUP serial engine's maximum frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

Typically, the following sensor hardware's maximum bus interface frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

Based on this example, following is the appropriate value for `max_bus_speed_khz` item:

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

8. If I^2C or SPI is interface used for the sensor, then the set driver registry configuration item `dri_irq_num` with the GPIO number is used for the interrupt.
9. Modify the `num_rail`, `vdd_rail`, and `vddio_rail` item values according to the power rail(s) required for the sensor.
   a. Set the `num_rail` item value to denote the total number of power rail(s) required for the sensor.
   b. Refer power-rail-and-name-mapping and note the power rail name to be used in the sensor registry.
   c. Based on the power rail(s) used for sensor, keep only the required `vdd_rail` and/or `vddio_rail` item in the configuration.

      For example, if the sensor uses a single power rail, then set the `num_rail` item value to `1` and keep only the applicable `vdd_rail` or `vddio_rail` item in the configuration.
10. After compiling the LPAIDSP code, verify that the driver registration function (see `register_func_name` value in the driver scons file to know the registration function name) is added to `sns_static_drivers_list[]` in the `adsp_proc/qsh_`

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:41 GMT hyungchul.yoon@qc.com

`platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/sns_static_drivers.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that is different than the QTI reference design sensor types and if the sensor needs to be exposed to Sensor HAL, Android framework, and Android App then:

a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.

b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then refer the existing `electro_neuro_graph.cpp` or `ambient_light.cpp` file and create a new CPP file to support new sensor type.

2. If the driver has custom protobuf file (.proto), copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto`.

Verify sensor

Refer QSH sensor verification to verify the sensor.

Integrate third-party provided algorithm

Perform the following steps to integrate the third-party sensor drivers on LPAIDSP.

LPAIDSP side changes

1. Acquire the directory from vendor that contains the algorithm, associated SCons build file, any custom prototype file for algorithm, and JSON configuration files, if any.
2. Place the acquired directory in `adsp_proc/qsh_algorithms`.
3. If the driver has custom protobuf file (.proto), copy the file to `adsp_proc/qsh_api/pb`.
4. To enable the sensor driver for compilation, add driver SCons filename (without the SCons extension) into `include_qsh_algorithms_libs.extend` array in the `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.
5. To enable the Island support for the driver, refer the algorithm Scons file to find a flag with the `SNS_ISLAND_INCLUDE_` keyword. Then add the rule to enable the same flag into `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.

   For example: `env.AddUsesFlags(['SNS_ISLAND_INCLUDE_XXXX'])`

6. If the algorithm has any JSON config file:

   The Registry sensor will be enabled in future software release. Hence, the *Registry* sensor dependency must be disabled in the algorithm.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:36:44 GMT hyungchul.kim@lge.com

Ensure that the driver Scons file handles `SNS_DISABLE_REGISTRY` macro to disable the registry operations and then in the algorithm hardcode the values matching to the JSON config file. If the algorithm Scons file doesn't support `SNS_DISABLE_REGISTRY` macro, then receive a modified version of the driver without registry operations and with hardcoded values in the algorithm matching to the JSON config file.

7. After compiling the LPAIDSP code, verify that the algorithm registration function (see `register_func_name` value in the driver scons file to know the registration function name) is added to `qsh_static_algorithms_list[]` in the `adsp_proc/qsh_platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/qsh_static_algorithms.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that is different than the QTI reference design sensor types and if the sensor needs to be exposed to Sensor HAL, Android framework, and Android App then:

a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.

b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then refer the existing `gravity.cpp` or `step_count.cpp` file and create a new CPP file to support the new sensor type.

2. If the algorithm has custom protobuf file (.proto), copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto`.

Verify sensor

Refer QSH sensor verification to verify the sensor.

Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 03:30:44 GMT hyungseul.wong@qc.com

Develop algorithm

See the QSH APIs mentioned in QSH APIs and the sample algorithms mentioned in Sample algorithm to develop a new algorithm.

9.4 QSH sensor verification

Use one of the following methods to verify the sensor functionality.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

Verification	Sensor availability	Sensor data flow
Method 1: Using Android App	Open Qsensortest APK and verify whether required sensor type is listed.	Tap on the sensor type entry to enable sensor and observe the data received/displayed. To disable sensor, click the red **X** icon.

Method 2: Using native test tools	Run `ssc_sensor_info` command to check if the sensor type is listed.
	`adb root`
	`adb shell ssc_sensor_info`
		Run `see_workhorse` command to enable sensor for certain duration and observe the data received/displayed.
		For streaming sensor: `adb shell see_workhorse -sensor=<sensor_type> -sample_rate=10 -duration=10 -display_events=1`
		Example: `adb shell see_workhorse -sensor=accel -sample_rate=10 -duration=10 -display_events=1`
		For on-change sensor: `adb shell see_workhorse -sensor=<sensor_type> -on_change=1 -duration=10 -display_events=1`
		Example: `adb shell see_workhorse -sensor=ambient_light -on_change=1 -duration=10 -display_events=1`

Method 3: Using USTA	Refer Unified Sensor Test Application (USTA).	

9.5 RSB integration

This section will be updated in a future release, after the software support is enabled for RSB.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

9.6 Sensor axis mapping and calibration

See Sensor axis mapping and calibration (*KBA-190307232731*) for axis mapping and calibration steps for each sensor type.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

10 Memory usage

This chapter provides methods to identify memory usage of particular module part of SWM and LPAIDSP image.

10.1 SWM image memory usage

The memory report generated in SWM is based on the facilities of Zephyr build system.

SWM memory report file is `wm_proc/config/bsp/lpai_swm_img/build/aspen.swm.prod/LPAI_SWM_IMG_aspen.swm.prod_memory_report.txt`.

10.2 LPAIDSP image memory usage

LPAIDSP Sensor PD memory report files are generated at `adsp_proc/config/bsp/sensor_img/build/aspen.adsp.prod`.

- `Memory_info_aspen_SensorsPD.csv`: Provides information about the non-Island modules.
- `Memory_info_aspen_SensorsPD_island.csv`: Provides information about the Island modules.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:35:44 GMT hyunchul.wonn@lge.com

11 Tools to test sensors

11.1 Android applications

Unified Sensor Test Application (USTA)

- USTA bypasses sensors HAL and directly communicates with QSH framework on SWM and LPAIDSP.
- USTA can be useful in early sensor bring-up phase where particular sensor isn't exposed in sensor HAL.
- USTA could send requests to respective sensors and decode received events.
- USTA supports the custom proto file.
- On wearables platform, USTA is only supported in command line mode.
- For more information on USTA, see Unified Sensor Test Application (USTA) User Guide (80-P9301-85).

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

QSensorTest Application (QSTA)

- QSTA uses standard Android framework APIs to enable/disable sensors, hence all requests going through sensors HAL.
- On wearables platform, QSTA is only supported in command line mode.
- QSTA command line parameters:

Parameter	Purpose
sensorname	Name of the sensor
sensortype	Type of the sensor
rate	Sample rate in milliseconds for streaming sensor
batch	Batch period in milliseconds

- The following commands are used to enable/disable sensors via command line.

1. **Start QSTA service**

```
adb shell am start-foreground-service com.qualcomm.
qti.sensors.qsensortest/com.qualcomm.qti.sensors.
core.qsensortest.QSensorService
```

2. **Get list of all sensors**

```
adb shell sqlite3 /data/data/com.qualcomm.qti.
sensors.qsensortest/databases/qsensortest.db
'select sensor_name from Sensor'
```

Output format: `<sensor_name>.<sensor_type>`

Sample output:

```
lsm6dsv Accelerometer Non-wakeup.1
pedometer_wrist Non-wakeup.19
```

3. **Prepare command parameters**

   A list of all the sensors is displayed in the previous step. Following format shows how to identify sensor name and sensor type in the output.

   According to the sample output in the previous step, the sensor name is `lsm6dsv Accelerometer Non-wakeup` and the sensor type is `1`.

   To enable/disable the sensor:

   – Replace all the white spaces in sensor name and use it as a value for the `sensorname` parameter.

   – Use the sensor type as a value for the `sensortype` parameter.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 02:38:34 GMT hyungchul.won@qc.com

–**Streaming sensor:**

- To enable: Use a non-zero sample rate and batch period as desired.
- To disable: Use `-1` as the sample rate.

–**On-change sensor:**

- To enable: Use zero sample rate and batch period as desired.
- To disable: Use `-1` as the sample rate.

4. **Enable sensor**

For streaming sensor

```
adb shell am broadcast -a com.qualcomm.qti.sensors.qsensortest.
intent.STREAM -e 'sensorname' 'lsm6dsv_Accelerometer_Non-wakeup'
--ei sensortype 1 --ei rate 60 --ei batch 60
```

For on-change sensor

```
adb shell am broadcast -a com.qualcomm.qti.sensors.qsensortest.
intent.STREAM -e 'sensorname' 'pedometer_wrist__Wakeup' --ei
sensortype 19 --ei rate 0 --ei batch 0
```

5. **Disable sensor**

For streaming sensor

```
adb shell am broadcast -a com.qualcomm.qti.sensors.qsensortest.
intent.STREAM -e 'sensorname' 'lsm6dsv_Accelerometer_Non-wakeup'
--ei sensortype 1 --ei rate -1
```

For on-change sensor

```
adb shell am broadcast -a com.qualcomm.qti.sensors.qsensortest.
intent.STREAM -e 'sensorname' 'pedometer_wrist__Wakeup' --ei
sensortype 19 --ei rate -1
```

6. **Stop QSTA service**

```
adb shell am force-stop
com.qualcomm.qti.sensors.qsensortest
```

11.2 Android native SEE test tools

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@rse.com

ssc_sensor_info

This tool is used to list available QSH sensors. Use the following command to execute the tool.

```
adb root
adb shell ssc_sensor_info
```

see_workhorse

This tool is used to list and test the available QSH sensors.

To know more about this tool, print the tool options using `-h` command line parameter or see *Test tools* section on page 117 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35).

ssc_drva_test

Note: This tool will be functional in future software release.

This tool executes various sensor use cases including the few factory software and hardware tests.

To know more about this tool, see *Test tools* section on page 112-114 of Sensors Execution Environment (SEE) Sensors Deep Dive (80-P9301-35).

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:39:44 GMT hyungchul.won@lge.com

12 Sensors vendor ecosystem

This chapter provides information about the sensor vendor ecosystem for sensor parts and algorithms different than provided by Qualcomm on SW6100 QTI platform.

- QTI provides complete hardware and software ecosystem for sensor vendors to develop SEE drivers and algorithms.
- **For sensor driver**
  - Hardware/Software platform: HDK8650 or later with OpenSSC 12.x package or later.
  - Qualcomm recommends OEM to request vendor for the PVL validation.
  - Validation and delivery: The sensor vendors can provide drivers to customers directly after complete validation. OEM can request the following from vendors for a driver delivery:
    - QTI-defined Driver Acceptance checklist listing the passing and failing test cases.
    - CTS test results.

Note: QTI is not committed to provide any support for issues that are related to, or a result of, use of sensor vendor drivers other than provided by in the mainline software, and OEM should contact vendor for those issues. OEMs need to ask sensor vendors to provide fully validated drivers on the recommended HDK/OpenSSC platform and have sensor part fully tested, certified.

- **For sensor algorithm** - Contact QTI.

Note: QTI is not committed to provide any support for issues that are related to, or a result of, use of sensor vendor algorithm, on SW6100 QTI platform, and OEM should contact vendor for those issues. OEMs need to ask sensor vendors to provide validated algorithm on the recommended HDK/OpenSSC platform or QTI reference device.

Qualcomm Confidential - May Contain Trade Secrets 2025-03-27 02:30:44 GMT hyungchulw@qlge.com

13 Build and sideloading

This chapter provides references to software build steps and image sideloading procedure for SWM and LPAIDSP.

13.1 Build SWM and LPAIDSP

Refer release notes to know the host machine build environment and build commands for SWM and LPAIDSP software.

13.2 Sideload SWM and LPAIDSP

See *Sideloading the images* section of SW6100 Software User Guide (80-74841-50), to know the procedure to sideload SWM and LPAIDSP images.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hyungchul.won@lge.com

14 Debug methods

This chapter describes the debug methods available for sensor issues. The following table depicts debug method per activity.

Debug activity	Debug method
Sensor HAL logs	Logcat
SWM QSH logs	• QXDM Professional • On-device Diag logging • Zephyr shell
LPAIDSP QSH logs	• QXDM Professional • On-device Diag logging
Crashes and complex issues on LPAIDSP and SWM	Ramdump

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:41 GMT hyunchul.won@lge.com

14.1 Sensor HAL logs

Teh sensor HAL default logcat logging level is set to `INFO` level. To enable the `VERBOSE` level logging, run the following command and reboot the device.

```
adb root
adb shell setprop persist.vendor.sensors.debug.hal v
adb reboot
```

14.2 QXDM Professional

Note: This method can be used for SWM logging in future software release.

- QXDM Professional supports live Diag logging via USB connection. Sensor modules can use the QSH provided macros and APIs to send debug print and log packets to QXDM

Professional.

- See the *QXDM logging* section of SW6100 Software User Guide (80-74841-50), to know the procedure for capturing the QXDM Professional log. While following the steps, use the following log masks for sensors related logs.
  - Message Packets → SNS → All logs
  - Log Packets → Common → SNS → All logs

14.3 On-device diag logging

Note: This method can be used for SWM logging in future software release.

- On-device logging allows diagnostic traffic to be saved as a file on the device. It can be used in field testing when connecting the device to a workstation or laptop is undesirable. Later, host PC tools can be used to process the saved log file.
- See the *On-device logging* section of SW6100 Software User Guide (80-74841-50), to know the procedure for capturing on-device diag logging. While performing the steps, use the following log masks for sensors related logs.
  - Message Packets → SNS → All logs
  - Log Packets → Common → SNS → All logs

14.4 Logging over Zephyr shell (UART)

See the *Zephyr shell logging* section of SW6100 Software User Guide (80-74841-50), to know the procedure to add log into Zephyr shell.

14.5 Ramdump

This method allows capturing dump of various memories, such as DDR, LPAIDSP TCM, and SWM TCM. The ramdump is then parsed to extract the debug information, such as call stack, and log buffer.

See the *RAM dump mode and RAM dump collection* section of SW6100 Software User Guide (80-74841-50), to know the procedure for capturing ramdump.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:44 GMT hy.nachuk.kim@lge.com

15 Sensors customer support

Customer engineering support is provided through the QTI Salesforce Portal, https://support.qualcomm.com/.

For QSH Sensors related support, select the problem areas to direct the case to the appropriate Sensors team.

- Problem Area 1
  - Board Support Package (BSP)
- Problem Area 2
  - Sensors
- Problem Area 3
  - **For AP side issues**: Sensors – AP/HLOS
  - **For non-AP side issues**: Sensors Core – SSC

- For QSH context aware sensors, select the following problem areas to direct the case to the appropriate technology team.

Technology	Audio	Location	BLE	Wi-Fi	WWAN
Problem Area 1	Multimedia	Wireless Connectivity Software			Modem Software
Problem Area 2	Audio (Multimedia)	gpsOne	Bluetooth	WLAN	QMI/RIL/Telephony/Android and Windows Data
Problem Area 3	Choose according to the observed issue				Android RIL

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 02:30:14 GMT hwingawl.won@me.com

16 References

16.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
Sensors Execution Environment (SEE) Sensors Deep Dive	80-P9301-35
Sensors Execution Environment Client API	80-P9301-36
Adding a Custom Sensors Algorithm with Sensors Execution Environment (SEE)	80-P9301-67
Unified Sensor Test Application (USTA) User Guide	80-P9301-85
Qualcomm® Sensing Hub (QSH) 4.0 API Reference	80-40939-19
SW6100 Software User Guide	80-74841-50
SW6100 Device Interfaces Guide	80-74841-87

Qualcomm
Confidential - May Contain Trade Secrets
2020-02-27 02:30:44 GMT
hyungchul.won@lge.com

16.2 Acronyms and terms

Sensors-related acronyms and expansions for better understanding.

Acronym or term	Definition
ALS	Ambient light sensor
AM	Ambient microcontroller
ASCP	Asynchronous COM port
DRI	Data ready interrupt
DSP	Digital signal processor
ECG	Electrocardiogram
EIS	Electronic image stabilization
ENG	Electroneurography
GRV	Game rotation vector
HLOS	High level operating system
I^2C	Inter-integrated circuit
I^3C	Improved I^2C

Acronym or term	Definition
IBI	In-band interrupt
IMU	Inertial measurement unit
ODR	Output data rate
PD	Protection domain
PPG	Photoplethysmogram
QSH	Qualcomm sensing hub
QuP	Qualcomm universal peripherals
QuRT	Qualcomm real time operating system
RTOS	Real time operating system
SCP	Synchronous COM port
SCons	Software construction
SMP2P	Shared memory point to point
SoC	System-on-chip
SPI	Serial peripheral interface
SSC	Qualcomm® Snapdragon™ sensors core
SUID	Sensor unique identifier
SWM	Sensors Wearables Microcontroller
TCM	Tightly coupled memory
UART	Universal asynchronous receiver/transmitter
WM	Wearables Microcontroller

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 02:30:44 GMT
hyungchul.won@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Qualcomm
Confidential - May Contain Third Parties
2026-02-27 02:30:44 GMT
hyungchul.woo@lge.com